

实验二报告

唐宇恩 251220100

2026 年 5 月 3 日

1 3-8 译码器

1.1 整体方案设计

端口定义:

端口符号	端口功能
输入端 A, B, C	从低到高的二进制数
输入端 $G1, G2A_L, G2B_L$	控制端
输出端 $Y0_L \sim Y7_L$	从 0 到 7 的十进制数输出端，均低电平有效

1.2 实验电路图

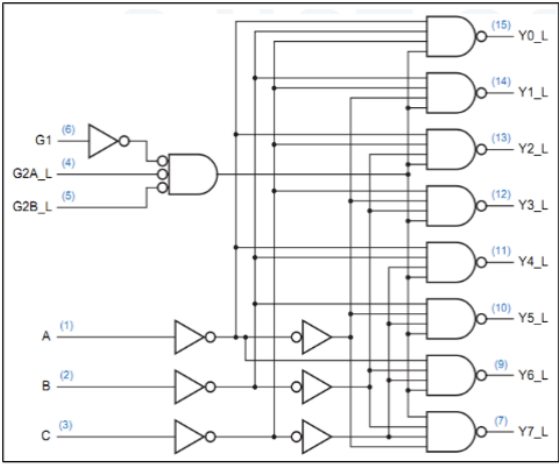


图 1: 3-8 译码器原理图

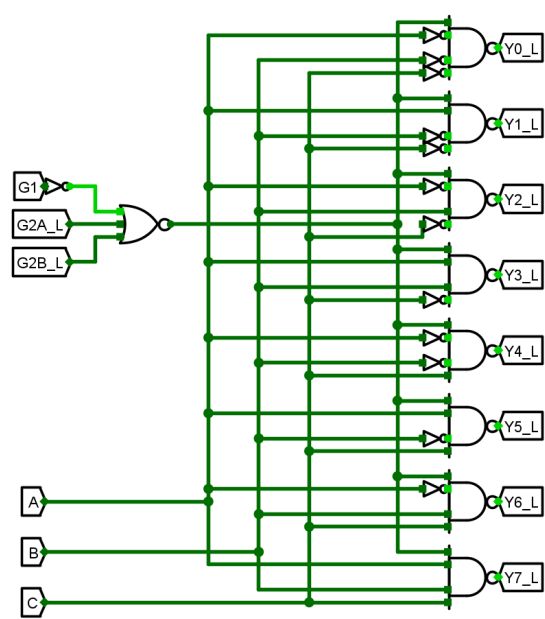


图 2: 电路图

1.3 仿真测试

表 1: 74X138 功能仿真结果

G1	G2A_L	G2B_L	C	B	A	Y7	Y6	Y5	Y4	Y3	Y2	Y1	Y0
X	1	X	X	X	X	1	1	1	1	1	1	1	1
X	X	1	X	X	X	1	1	1	1	1	1	1	1
0	X	X	X	X	X	1	1	1	1	1	1	1	1
1	0	0	0	0	0	1	1	1	1	1	1	1	0
1	0	0	0	0	1	1	1	1	1	1	1	0	1
1	0	0	0	1	0	1	1	1	1	1	0	1	1
1	0	0	0	1	1	1	1	1	0	1	1	1	1
1	0	0	1	0	0	1	1	0	1	1	1	1	1
1	0	0	1	0	1	1	1	0	1	1	1	1	1
1	0	0	1	1	0	1	0	1	1	1	1	1	1
1	0	0	1	1	1	0	1	1	1	1	1	1	1

2 8-3 优先级编码器

2.1 整体方案设计

端口定义：

端口符号	端口功能
输入端 $I_0 I_7$	从 0 到 7 的十进制数输入端，优先级为 $I_0 > I_1 > \dots > I_7$
输出端 $O_0 O_2$	从低到高的二进制输出端

2.2 实验电路图

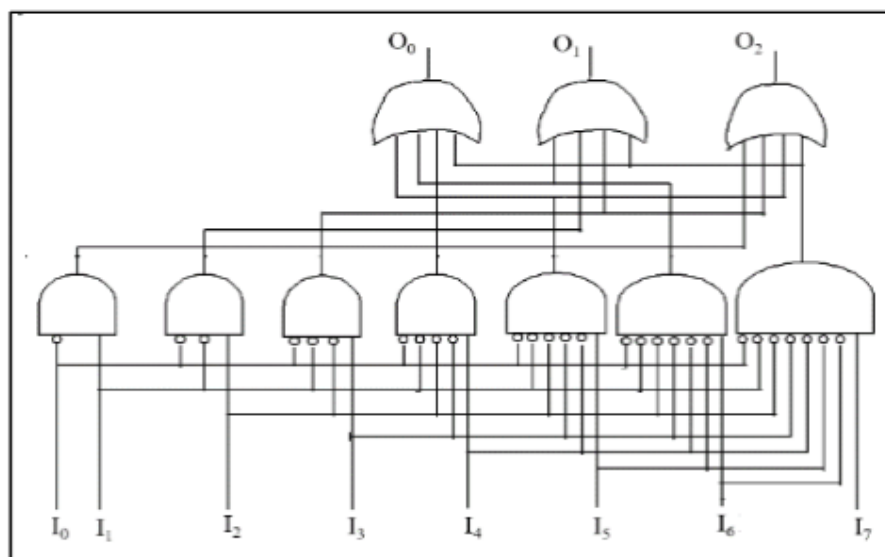


图 3: 8-3 优先级编码器原理图

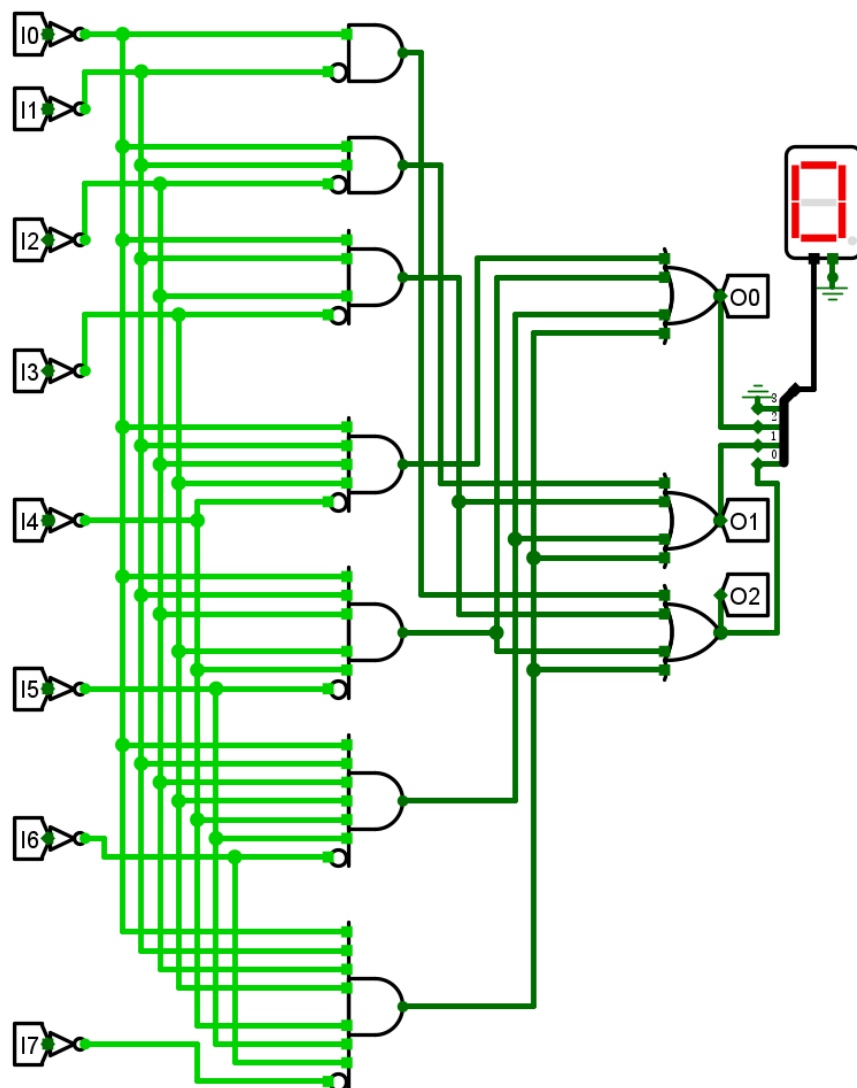


图 4: 电路图

2.3 仿真测试

表 2: 8-3 优先级编码器功能仿真结果											
I0	I1	I2	I3	I4	I5	I6	I7	O2	O1	O0	Hex 显示
1	X	X	X	X	X	X	X	0	0	0	0
0	1	X	X	X	X	X	X	0	0	1	1
0	0	1	X	X	X	X	X	0	1	0	2
0	0	0	1	X	X	X	X	0	1	1	3
0	0	0	0	1	X	X	X	1	0	0	4
0	0	0	0	0	1	X	X	1	0	1	5
0	0	0	0	0	0	1	X	1	1	0	6
0	0	0	0	0	0	0	1	1	1	1	7

3 4 位串行加减法器

4 整体方案设计

端口定义:	
端口符号	端口功能
输入端 $X_0 X_3$	加数/被减数的 4 位二进制输入端
输入端 $Y_0 Y_3$	加数/减数的 4 位二进制输入端
输入端 Cin	进位输入端, 控制加法/减法操作
输出端 $S_0 S_3$	结果的 4 位二进制输出端
输出端 $Cout$	表示加法/减法结果的进位/借位情况

4.1 实验电路图

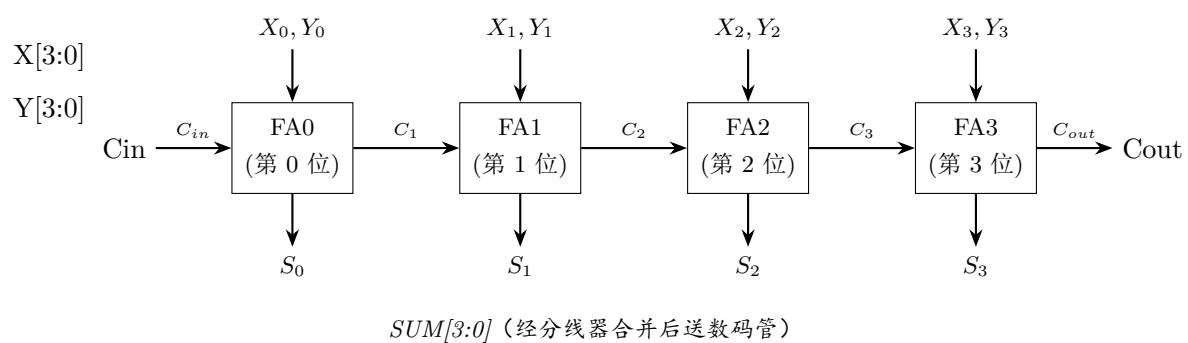


图 5: 4 位串行进位加法器顶层模块图

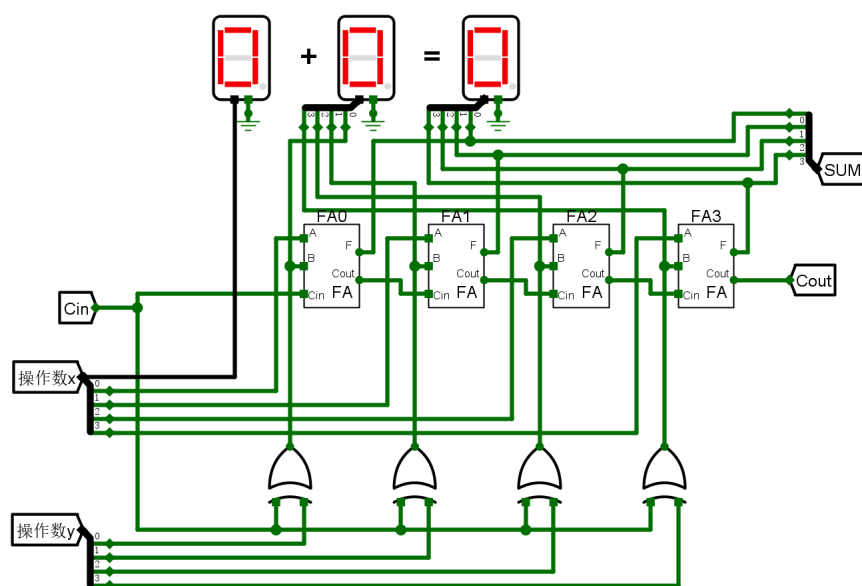


图 6: 主电路图

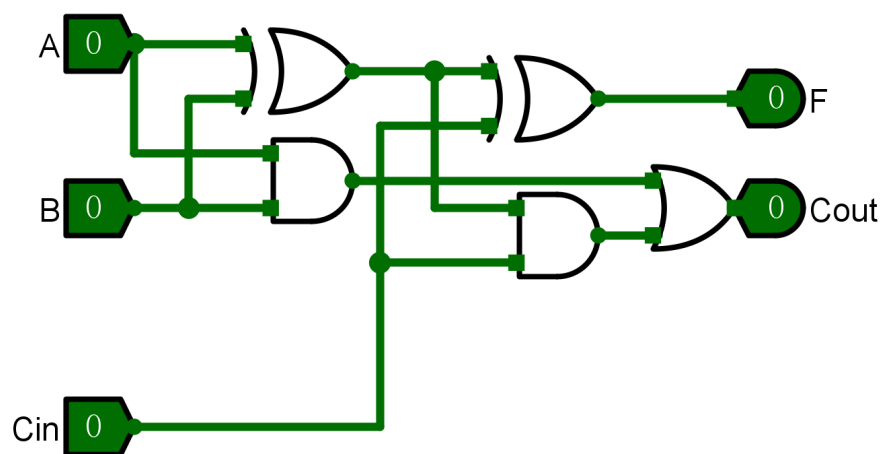


图 7: FA 子电路图

4.2 错误分析

评测时遇到减法全挂的现象，原因是未能准确理解实验平台的实验说明中“借位输出”的含义，误认为是当输出端 $Cout$ 为 1 时表示有借位，实际上实验平台上 $Cout$ 为 0 时表示有借位。将电路中原先存在的 $Cin \oplus$ 最后一位 $Cout \rightarrow Cout$ 改为 最后一位 $Cout \rightarrow Cout$ 后，评测得以通过。希望实验平台上的实验说明能够明确说明借位输出的逻辑状态，以避免类似的误解。

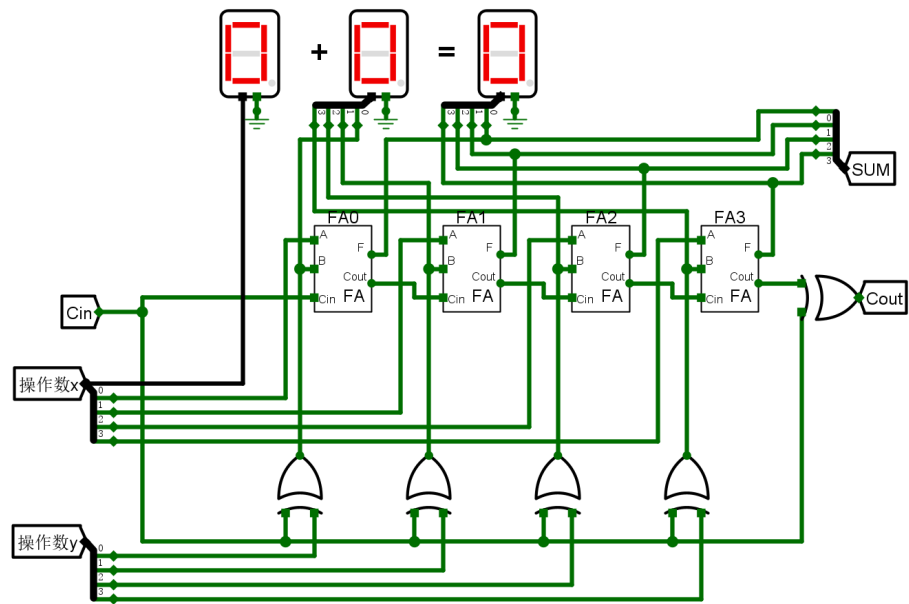


图 8: 错误电路图

测试说明

请不要更改电路封装、修改引脚、添加引脚，否则评测将出错。

注意：。

同理，X,Y表示对4位加减法器的输入，Cin表示加减法运算选择，Sum表示加减法器的结果输出，Cout表示加减法器的进位/借位输出。评测真值表以16进制表示。

本关的测试数据很多。

平台会对你编写的代码进行测试：

图 9: 平台上不够明确的说明

5 7 位汉明码纠错电路

5.1 整体方案设计

端口定义：

端口符号	端口功能
输入端 <i>Input</i> [0 : 7]	8 位数据输入端
输出端 <i>Output</i> [0 : 7]	8 位数据输出端，纠错后的数据

电路由三部分组成：

1. 奇偶校验子电路：3 个 4 输入奇偶校验器（异或链），分别计算 S_1 、 S_2 、 S_3 ；

2. **3-8 译码器**：以 $S_3S_2S_1$ 为地址，输出低有效的 $Y_0 \sim Y_7$ ，经取反后得到对应位的纠错使能信号；
3. **异或纠错门**：每个数据/校验位与对应的译码器输出（已取反为高有效）异或，实现单比特翻转纠错。

实验原理图：

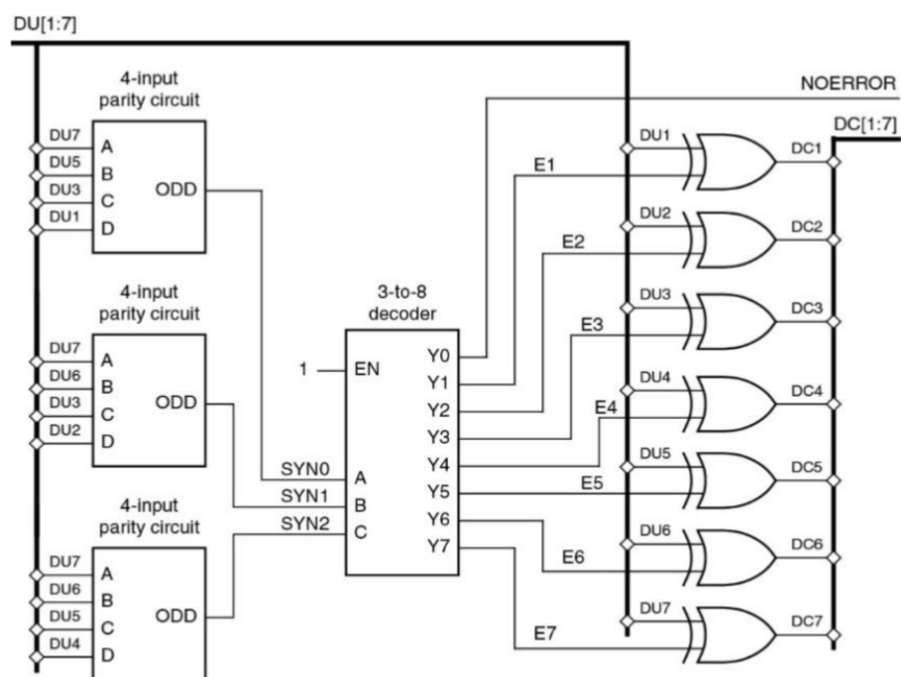


图 10: 7 位汉明码纠错电路原理图

5.2 实验电路图

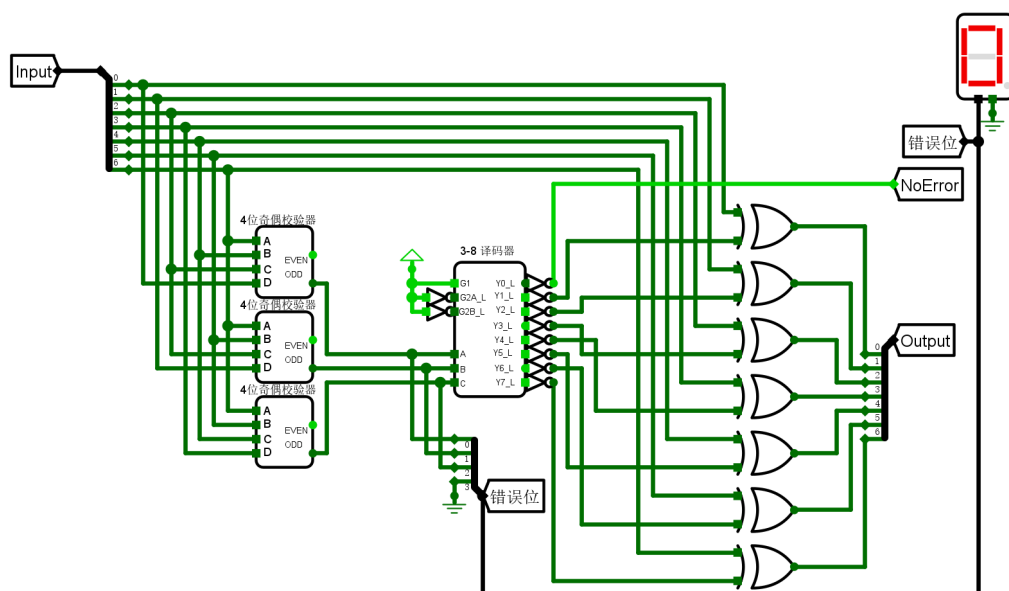


图 11: 主电路图

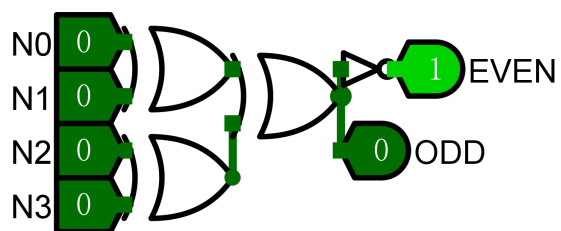


图 12: 奇偶校验子电路图

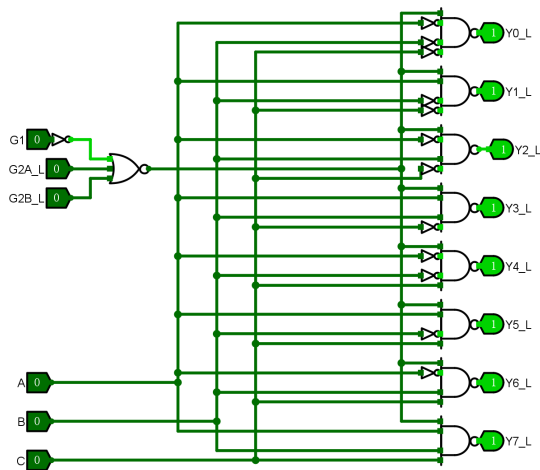


图 13: 3-8 译码器电路图

6 8 位桶形移位器

6.1 整体方案设计

端口定义：

端口符号	端口功能
输入端 $din[0 : 7]$	8 位数据输入端
输入端 $shamt[0 : 2]$	移位量输入端
输入端 L/R	移位方向控制端，1 表示左移，0 表示右移
输入端 A/L	算术/逻辑移位控制端，1 表示算术移位，0 表示逻辑移位
输出端 $dout[0 : 7]$	8 位数据输出端

其中，L/R 用于选择移位方向，A/L 用于选择移位类型：当进行右移时，A/L=1 表示算术右移，高位补符号位；A/L=0 表示逻辑右移，高位补 0。左移时，A/L 主要用于标明逻辑/循环移位模式，原理图中用独立控制端刻画其作用。

实验原理图：

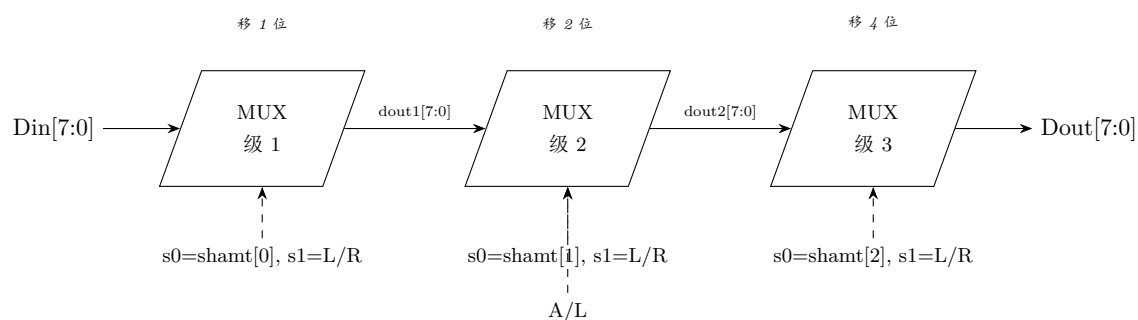


图 14: 8 位桶形移位器三级 MUX 原理图

每级 4 路 8 位选择器的选择逻辑如下：

- $s_0 = 0$ （不移位）：直接传递当前数据；
- $s_0 = 1, s_1 = 0$ （右移）：数据右移对应位数，高位补 0（逻辑）或符号位 $\text{din}[7]$ （算术），由 A/L 控制；
- $s_0 = 1, s_1 = 1$ （左移）：数据左移对应位数，低位补 0（逻辑）或移出位（循环），由 A/L 指示移位方式。

6.2 实验电路图

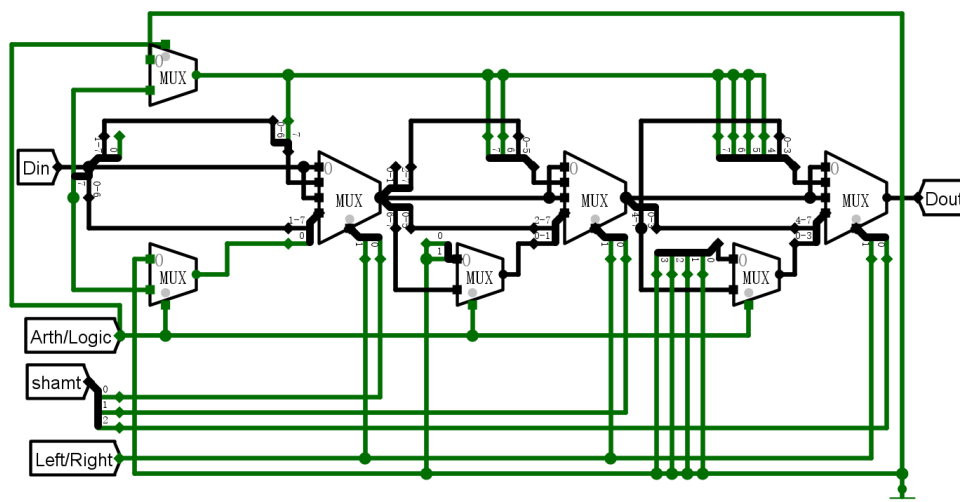


图 15: 8 位桶形移位器电路图

7 思考题

7.1 加减法器标志位

在 4 位串行进位加法器基础上，增加以下组合逻辑生成标志位：

- 进位标志 **CF** (Carry Flag)：直接取最高位全加器 FA3 的进位输出 Cout：

$$CF = C_{out}$$

- 溢出标志 **OF** (Overflow Flag)：有符号数溢出，当最高位进位与次高位进位不一致时溢出：

$$OF = C_{out} \oplus C_3$$

其中 C_3 为 FA3 的进位输入（即 FA2 的进位输出）。

- 符号标志 **SF** (Sign Flag)：结果的最高位（符号位）：

$$SF = S_3$$

- 零标志 **ZF** (Zero Flag)：结果全为 0 时置 1，可用 4 输入或非门实现：

$$ZF = \overline{S_3 + S_2 + S_1 + S_0} = \overline{S_3} \cdot \overline{S_2} \cdot \overline{S_1} \cdot \overline{S_0}$$

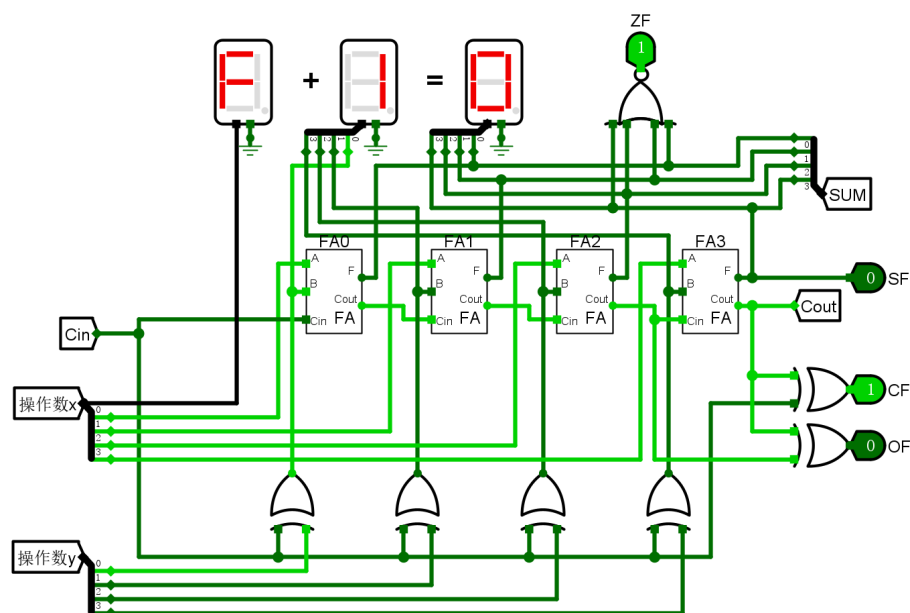


图 16: 加减法器标志位

7.2 比较信号

当对带符号整数 X, Y 进行比较时, 采用 $OF \oplus SF$ 时表示 $X < Y$, 采用 $\overline{ZF} \wedge \overline{SF \oplus OF}$ 时表示 $X > Y$ 。

当无符号整数 X, Y 进行比较时, 采用 CF 时表示 $X < Y$, 采用 $\overline{CF}$ 时表示 $X > Y$ 。

7.3 4 位无符号二进制数比较器

对于无符号 4 位数 $X = X_3X_2X_1X_0$ 与 $Y = Y_3Y_2Y_1Y_0$, 从最高位向最低位逐位比较:
大于 ($X > Y$) 的逻辑表达式:

$$X > Y = X_3\overline{Y_3} + E_3\overline{X_2Y_2} + E_3E_2\overline{X_1Y_1} + E_3E_2E_1\overline{X_0Y_0}$$

其中 $E_i = \overline{X_i \oplus Y_i}$ (第 i 位相等标志)。

小于 ($X < Y$) 的逻辑表达式:

$$X < Y = \overline{X_3}Y_3 + E_3\overline{X_2}Y_2 + E_3E_2\overline{X_1}Y_1 + E_3E_2E_1\overline{X_0}Y_0$$

电路图如下:

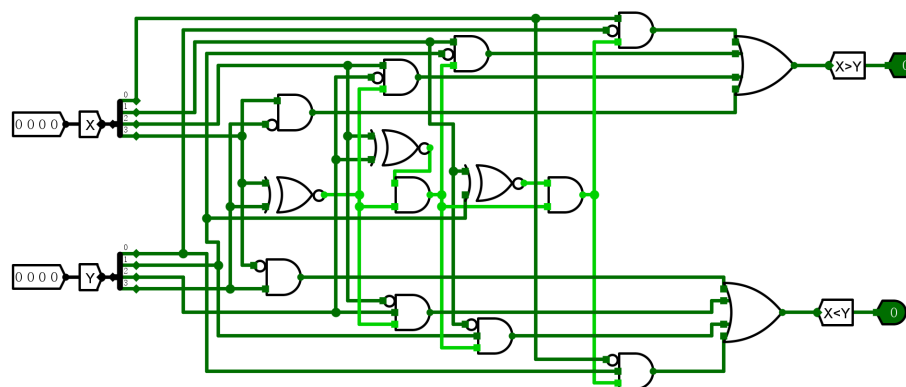


图 17: 4 位无符号二进制数比较器电路图

7.4 32 位桶形移位器

32 位桶形移位器需要使用 5 位 shamt 输入。如果希望复用 8 位桶形移位器，反而会增加复杂度，需要对 8 位移位器增加移入和移出位来支持移位器间的传递，且每级 MUX 的输入选择逻辑会更复杂。直接仿照 8 位设计一个 32 位的三级 MUX 结构更为清晰和高效。每级 MUX 的选择逻辑与 8 位版本类似，只是输入和输出宽度增加到 32 位，移位量控制信号也相应增加到 5 位（shamt[0:4]）。

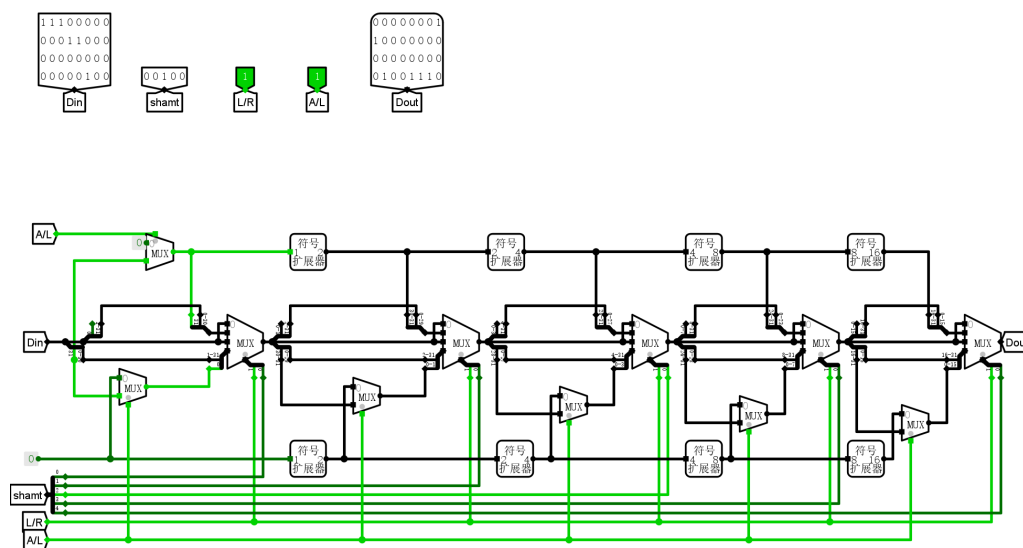


图 18: 32 位桶形移位器原理图